

Algorithms for 1-Norm-Oriented Search of Commuting Pauli Generators

1 Problem statement

Consider an N -qubit Hamiltonian written as a Pauli expansion

$$H = \sum_{i=1}^m c_i P_i, \quad c_i \in \mathbb{R}, \quad P_i \in \mathcal{P}_N, \quad (1)$$

where $\mathcal{P}_N$ is the N -qubit Pauli group modulo phases. We would like to choose $r = N/2$ independent commuting Pauli generators

$$G_1, \dots, G_r, \quad (2)$$

so that the part of H commuting with all of them is as large as possible in coefficient 1-norm.

Let

$$\mathcal{S} = \langle G_1, \dots, G_r \rangle \quad (3)$$

be the Abelian subgroup generated by the chosen Pauli products. The natural objective is not the abstract size of the commutant, because for fixed rank r the size of the centralizer in the Pauli group depends only on r , not on the particular commuting subgroup. The nontrivial quantity is instead the captured Hamiltonian weight

$$W(\mathcal{S}) = \sum_{i=1}^m |c_i| \mathbf{1}[P_i \in \mathcal{C}(\mathcal{S})], \quad (4)$$

where

$$\mathcal{C}(\mathcal{S}) = \{Q \in \mathcal{P}_N : [Q, G] = 0 \text{ for all } G \in \mathcal{S}\} \quad (5)$$

is the centralizer of $\mathcal{S}$ inside the Pauli group. The optimization problem is therefore

$$\max_{\mathcal{S} \text{ Abelian, rank}(\mathcal{S})=N/2} W(\mathcal{S}). \quad (6)$$

This note summarizes exact algorithms for (6), together with practical heuristics that make the search useful for larger Hamiltonians.

2 Binary symplectic formulation

A Pauli product on N qubits can be represented by a binary symplectic vector

$$p = (x \mid z) \in \mathbb{F}_2^{2N}, \quad (7)$$

where $x, z \in \mathbb{F}_2^N$ encode the X and Z supports. Two Pauli products $P(p)$ and $P(q)$ commute if and only if their symplectic pairing vanishes:

$$p \odot q := x \cdot z' + z \cdot x' = 0 \pmod{2}, \quad q = (x' \mid z'). \quad (8)$$

A commuting generator set $\{G_1, \dots, G_r\}$ therefore corresponds to an *isotropic subspace*

$$S = \text{span}_{\mathbb{F}_2}\{g_1, \dots, g_r\} \subset \mathbb{F}_2^{2N}, \quad g_a \odot g_b = 0, \quad (9)$$

with $\dim S = r$. The centralizer is the symplectic orthogonal complement

$$S^\perp = \{p \in \mathbb{F}_2^{2N} : p \odot s = 0 \text{ for all } s \in S\}. \quad (10)$$

Thus the objective becomes

$$W(S) = \sum_{i=1}^m |c_i| \mathbf{1}[p_i \in S^\perp], \quad (11)$$

where p_i is the symplectic vector corresponding to P_i .

This reformulation is the main conceptual step: the search is an optimization over $N/2$ -dimensional isotropic subspaces of $\mathbb{F}_2^{2N}$.

3 Exact algorithms

3.1 Branch-and-bound over isotropic subspaces

The most direct exact strategy is to build S one generator at a time,

$$S_0 = \{0\} \subset S_1 \subset \dots \subset S_r, \quad \dim S_k = k, \quad (12)$$

where each extension is obtained by adding a new symplectic vector $g \in S_k^\perp \setminus S_k$ so that $S_{k+1} = S_k + \text{span}\{g\}$ remains isotropic.

At a partial subspace S_k , the surviving Hamiltonian weight is

$$W(S_k) = \sum_{i=1}^m |c_i| \mathbf{1}[p_i \in S_k^\perp], \quad (13)$$

and this also gives an upper bound on all descendants:

$$W(S_{k'}) \leq W(S_k), \quad k' \geq k. \quad (14)$$

A branch-and-bound tree can therefore be explored by repeatedly

1. selecting a partial subspace S_k ,
2. generating candidate extensions g ,
3. computing the child score $W(S_k + \text{span}\{g\})$,
4. pruning children whose upper bound does not exceed the best complete score found so far.

This method is exact, but in the worst case the number of isotropic subspaces grows exponentially with N .

3.2 Weighted MaxSAT, pseudo-Boolean, and MILP formulations

The same optimization can be phrased as a constrained discrete optimization problem. One introduces binary decision variables for the selected generators and hard constraints enforcing

1. pairwise commutation of the chosen generators,
2. linear independence of the chosen generators,
3. the target rank $r = N/2$.

Then for each Hamiltonian term one introduces a binary indicator stating whether that term commutes with all selected generators. The objective is to maximize the weighted sum of these indicators with weights $|c_i|$.

This leads naturally to weighted MaxSAT, pseudo-Boolean, or MILP formulations. Such approaches are attractive because they can exploit mature solver technology and can be useful for moderate system sizes, but they remain exponential in the worst case.

3.3 Exact search on a truncated heavy core

A practically useful exact variant is to keep only a heavy subset of Hamiltonian terms accounting for a prescribed fraction of the total 1-norm, for instance 90% or 95%, and solve the exact problem only on this reduced set. If the coefficient distribution is strongly skewed, this can drastically reduce the effective problem size while preserving the most important physics in the objective.

4 Heuristic algorithms

For larger Hamiltonians, one typically gives up global optimality and instead searches for good isotropic subspaces using heuristic or hybrid methods.

4.1 Greedy growth by true marginal gain

A natural baseline is to build the commuting subgroup one generator at a time. Starting from $S_0 = \{0\}$, one chooses at step k a new generator $g \in S_k^\perp \setminus S_k$ maximizing the marginal gain

$$\Delta(g \mid S_k) = W(S_k + \text{span}\{g\}) - W(S_k). \quad (15)$$

Since adding generators can only shrink the centralizer, $\Delta(g \mid S_k) \leq 0$ in the definition above; equivalently one may maximize the retained weight

$$W(S_k + \text{span}\{g\}). \quad (16)$$

This is preferable to a prefix-based strategy that only asks which heavy terms commute with an initial coefficient-sorted subset, because the latter is only a surrogate for the true objective (11).

The main weakness of greedy growth is myopia: an early generator can block better solutions later.

4.2 Beam search

Beam search is often the most effective compromise between pure greedy selection and full exponential search. Instead of keeping only one partial subspace at depth k , one keeps the best B candidates according to a score such as

$$\text{score}(S_k) = W(S_k) \tag{17}$$

or a more refined upper bound that estimates how much additional weight may still survive after completion to rank r .

At each depth, every retained S_k is extended by a shortlist of candidate generators, and only the best B children are kept. If the beam width B is fixed, the runtime is polynomial in m and N up to the candidate-generation cost. As B grows, the method interpolates between greedy search and exhaustive search.

4.3 Local improvement by swaps

Any constructive method can be improved by local search. Let

$$S = \text{span}\{g_1, \dots, g_r\} \tag{18}$$

be an initial solution. One then performs one-generator or two-generator updates:

1. remove one or two generators from the current set,
2. insert replacements that preserve commutation and independence,
3. accept the change if $W(S)$ increases.

Repeated until no improving move exists, this yields a locally optimal commuting subgroup. In practice, such swap-based refinement is cheap and often improves noticeably over plain greedy search.

4.4 Randomized multi-start search

Another simple and effective strategy is to run a constructive heuristic many times with randomized tie-breaking, random candidate subsampling, or random initialization, and then keep the best result. This reduces sensitivity to unlucky early choices and can be combined naturally with beam search or local improvement.

4.5 Heavy-core first, heuristic completion

A hybrid method particularly suited to chemistry Hamiltonians is:

1. sort terms by $|c_i|$,
2. retain a heavy core carrying most of the total 1-norm,
3. optimize exactly or with a strong heuristic on that core,
4. complete the subgroup to rank $N/2$ using a cheaper greedy rule,
5. refine with local swaps on the full Hamiltonian.

This often preserves the dominant contribution to the objective while controlling computational cost.

4.6 Candidate restriction and screening

The raw set of all Pauli strings is far too large to search directly. In practice, one should restrict the candidate generators to a manageable pool, for example

1. Pauli terms already appearing in H ,
2. products of a small number of heavy Hamiltonian terms,
3. low-weight Pauli strings,
4. Pauli strings consistent with additional structure such as spatial locality, particle-number conservation, or fermionic provenance under Jordan–Wigner.

Even though the global optimum might lie outside such a restricted pool, this type of screening is usually essential for scaling to larger systems.

5 Complexity and scaling

The difficulty of the problem lies not in checking commutation, which is easy in the symplectic representation, but in selecting the best rank- $N/2$ isotropic subspace.

- **Exact methods.** Exact branch-and-bound, MaxSAT, MILP, or exhaustive isotropic-subspace search scale exponentially with N in the worst case.
- **Heuristics.** Greedy, beam search with fixed beam width, local swap descent, and randomized multi-start can be implemented in time polynomial in m and N , provided the candidate pool is itself polynomially bounded.
- **Tradeoff.** Recovering guaranteed global optimality typically restores exponential scaling.

Thus the practically important question is not whether one can avoid exponential scaling altogether, but which heuristics preserve enough of the objective (11) to remain useful on realistic Hamiltonians.

6 Recommended practical workflow

A robust workflow for large Hamiltonians is the following.

1. Convert the Hamiltonian terms to binary symplectic vectors.
2. Restrict the candidate generator pool using weight, locality, or structural information.
3. Keep a heavy core of Hamiltonian terms carrying most of the total 1-norm.
4. Run beam search on the heavy core to construct a rank- $N/2$ isotropic subspace.
5. Complete or refine the solution on the full Hamiltonian using local generator swaps.
6. Benchmark the heuristic on small instances against exact branch-and-bound or solver-based optimization.

This pipeline separates the conceptually clean optimization problem from the engineering choices required for large systems.

7 Remarks on relation to occupation-parity searches

If one restricts the allowed generators to occupation-number or block-parity operators of fermionic origin, then the search space is much smaller and part of the problem collapses to linear algebra over $\mathbb{F}_2$. The fully general Pauli search is therefore more expressive, but also substantially harder. The methods discussed above should be viewed as tools for navigating this larger space when one is willing to leave the purely diagonal, occupation-based sector.

8 Summary

The search for $N/2$ commuting Pauli generators maximizing the commuting 1-norm of a Hamiltonian is most naturally formulated as a weighted optimization over isotropic subspaces in binary symplectic space. Exact algorithms such as branch-and-bound and solver-based formulations are available but scale exponentially in the worst case. For larger Hamiltonians, the most useful methods are heuristic: greedy growth by the true retained-weight objective, beam search, local swap refinement, randomized multi-start, and heavy-core truncation. The main practical challenge is to balance expressive power against the size of the candidate space.